ISTA 130

Python Potpourri and Indexing

To Do

Lots of stuff

Indexing

Variables

What are variables?

➔ Variables are names that refer to values.

How do we create a variable in our code?

➔ With an ***assignment statement***, e.g. `x = 3`

Example:

```
x = 3
y = x
x = 7
```

`y` refers to what value at this point?

➔ `3`

Augmented Assignment

```
x = 3
x = x + 4
```

`x` refers to what at this point?

➔ `7`

We have more convenient syntax to do the same thing:

➔ `x += 8 # Same thing as x = x + 8`

➔ We call this ***augmented assignment*** because we're combining assignment with something else (in this case, addition).

➔ `x` refers to what now?

- `15`

Math

What is PEMDAS?

➔ PEMDAS describes the order in which expressions containing more than one arithmetic operator are evaluated.

There are two more arithmetic operators that we need to know:

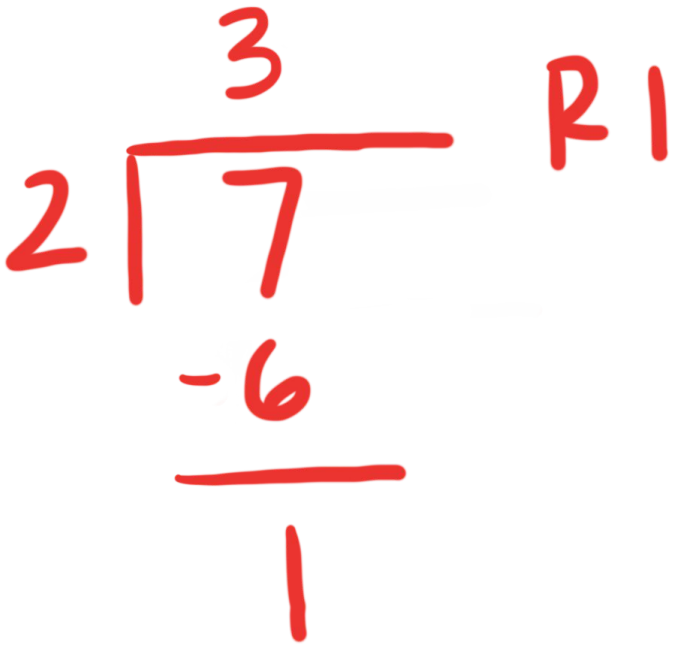
➔ `//` - integer division
➔ `%` - modulus (remainder)

These are like long division from elementary school.

The Modulus Operator: %

Just means find the remainder.

What is 7 divided by 2?



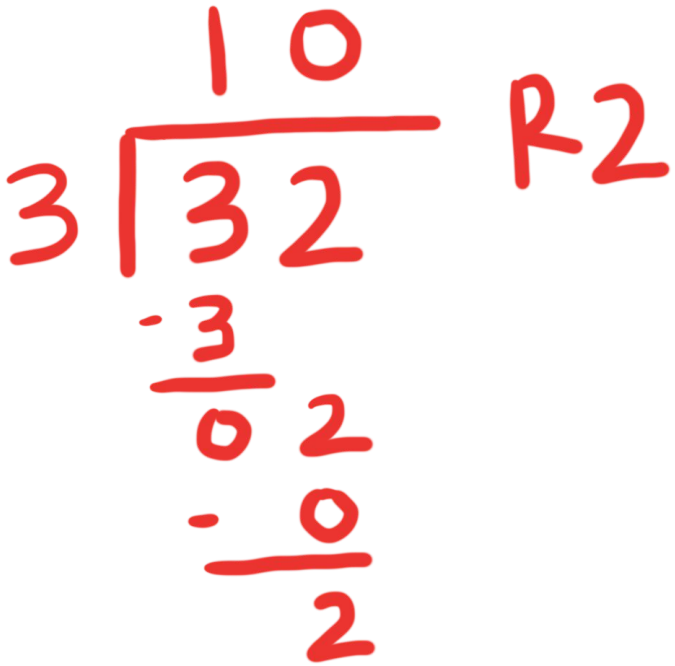

```
7 // 2 => 3

7 % 2 => 1
```

read this as "7 **mod** 2"

The Modulus Operator: %

What is 32 divided by 3?




```
32 // 3 => 10

32 % 3 => 2
```

What can we use it for?

What does it mean when:

- ➔ division by 2 leaves a remainder of 0?
- ➔ division by 2 leaves a remainder of 1?
- ➔ division by 5 leaves a remainder of 0?
- ➔ In general, when the remainder of an integer division is 0?

Let's review: the `print` function

`print` prints to the terminal/PowerShell window, i.e. so the human user can read it. It communicates to humans.

It takes zero or more arguments.

If an argument is not a string, it automatically converts the argument to one if it can.

It prints its arguments with spaces in between them (effectively replacing the commas in the argument list with spaces).

It always adds a newline character (therefore, calling it with no arguments prints a blank line).

In hw2, you used it to get information out of functions!

The `input` function - opposite of `print`

`input` gets text from a human typing at the keyboard.

It takes zero or one argument. If an argument is passed to input, it uses it has a prompt - it prints the argument message to the screen as an instruction to the user.

`input` ***always*** returns a string.

The `input` function - opposite of `print`

Try running this:

```
inches = input('Enter your height in inches: ')
feet = inches / 12.0
print('Your height in feet is:', feet)
```

What happened?

➔ After the input line, add this and run it again:
`print('Type:', type(inches))`

➔ So the data type of user input is `string`.

How can we tell Python that we want to get a `float` (or `int`) from the user?

`input` *always* returns a string

In order to convert our output to a `float` or `int`, we need to use a conversion function:

- ➔ `float()`
- ➔ `int()`
- ➔ `str()`

So input gives us a string, but we can convert strings to other types.

Here's the fix:

```
inches = input("Enter your height in inches: ")
inches = float(inches) # Convert the string to float
feet = inches / 12.0
print("Your height in feet is:", feet)
```

Now we're dividing a `float` by a `float`, no problems.

Trust the User?

Here's another issue: try running the fixed code but enter "coffee bean" at the prompt.

```
ValueError: could not convert string to float: 'coffee bean'
```

If we try to convert a string to another type, it won't work if there's no meaningful way to convert.

`for` Loops

What is the purpose of a `for` loop?

➔ To repeat code

What is the syntax for `for` loops we have met?

➔ A header with the syntax: `for i in range(some_num):`

➔ A body consisting of one or more Python statements indented one level more than the header.

What is `i` called in the header above?

➔ The ***loop control variable***.

`for i in range(some_num):`

How many times does the body of the loop execute?

➔ `some_num`. For example, if `some_num` is `4`, then the body of the loop will execute 4 times.

What value does `i` refer to initially?

➔ `0`.

What is the sequence of values that `i` takes on as the loop repeatedly executes?

➔ `0, 1,..., some_num - 1`

Can we use `i` in the body of the loop?

➔ Yes, it's just a variable that refers to an integer.

Accumulating Values into a Variable Inside a Loop

```
def sum_squares(max):
    '''
    Print the sum of squares of integers 1 to max.
    '''
    sum = 0
    print('x - x**2')
    print('--------')
    for i in range(max):
        num = i + 1
        square = num**2
        print(num, '-', square)
        sum = sum + square
    print()
    print('Sum of squares for 1 through', max, ":", sum)
```

Accumulating Values into a Variable Inside a Loop



It's like accumulating fish in a bucket:

```
bucket = bucket + fish
```

Accumulating Values into a Variable Inside a Loop

A two step process:

1. *Initialize* a variable *before* the loop.
2. *Update* the variable *inside* the loop.

Functions

What is a function?

➔ One or more Python statements packaged together and given a name so that the code can be executed by calling it by name,

What is the purpose of functions?

➔ ***Modularization***
- Makes code reusable
- Makes code clear and concise
- Makes code easy to maintain.
- Decomposes large, complex problems into smaller, easier problems.

Functions

How can we get information into functions?

➔ Argument-parameter passing process.

➔ Calls to the `input` function.

What is a parameter?

➔ A special kind of variable declared in the function signature as a placeholder. It has no value until the function is called and a corresponding argument is passed in.

How do we get information out of functions?

➔ So far, calls to the `print` function.

`return` Statements

`return` statements allow us to get information out of the function back to the calling code.

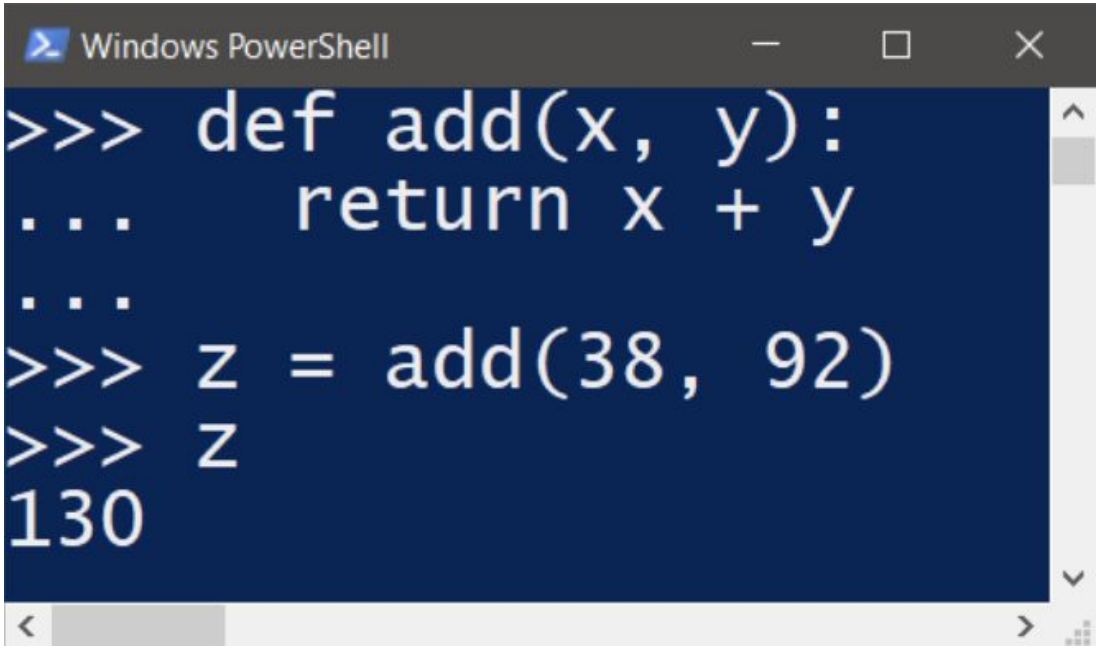
The syntax:

➔ `return expr`, where `expr` is a variable, arithmetic or other expression, or a literal.

When a function is executing after being called, as soon as Python encounters a `return` statement, the function **ends**.

The calling code captures the return value by assigning the function call to a variable.

Capturing the `return` value



How many times with the `for` loop execute?

```
def user_sum(n):
    '''
    Accumulate the sum of numbers 0 - n
    '''
    total = 0
    for i in range(n):
        input_num = input('Enter a number: ')
        total += int(input_num)
        return total
```

What we really wanted

```
def user_sum(n):
    '''
    Accumulate the sum of numbers 0 - n
    '''
    total = 0
    for i in range(n):
        input_num = input('Enter a number: ')
        total += int(input_num)
    return total
```

Strings

What is a string?

➔ A sequence of characters that Python uses to store text in a program.

How do we indicate a string literal in our code?

➔ We delimit the string with quotes.

What is string concatenation?

➔ Using the plus sign to build a new string from two pre-existing strings.

What is string repetition?

➔ Using the asterisk and an integer `n` to build a new string that consists of a pre-existing string repeated `n` times.

Strings

Concatenation:

```
>>> a = "The"
>>> b = "cat"
>>> ab = "The" + " " + "cat"
>>> ab
'The cat'
```

Repetition:

```
>>> b * 3
'catcatcat'
```

Strings

There is a built-in Python function `len` that returns the number of characterds in a string:

➔ `len('I heart Freya')` returns `13`

There is a string of length zero called the *empty string*. It is denoted by two quotes with nothing in between them, i.e. `''` or `""`.

➔ The equivalent of `0` for strings.
➔ We usually use this as the initial value when building a string in a loop.

Strings are ***immutable***, i.e., they can't be changed, only replaced.

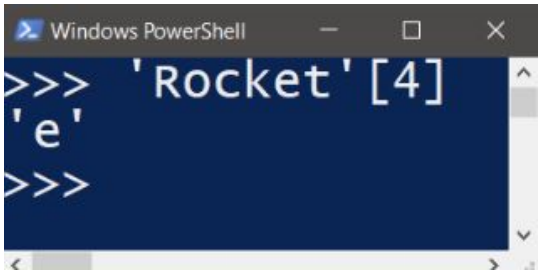
String Indexing

Strings are a sequence, which means that the elements of the string (the characters) have order.

Therefore, Python can refer to the positions of the characters using integers. These numbers start at zero and are called indices:



We can use the indexing operator, `[]`, to grab a copy of the character at a given position:



Do not assign to a position!

Exercises

Exercise: Apples

// does integer division
% gets the remainder

Let's use integer division and the modulus operator to solve a simple story problem:

➔ A family receives a box of apples in the mail.

Rather than bicker, they decide to divide the apples up evenly , and feed any remaining apples to Wilbur, their pet pot-bellied pig.



➔ Write a function called **`fruit_distributor`** that takes two parameters. The first is an integer for the number of pieces of fruit. The second is an integer for the number of people to distribute the fruit among. The function prints out the number of pieces per person and the number left over.

Exercise: combining `input`, `print`, `//`, and `%`

Write a function called **`height`** that gets the user's height in inches (as input) and prints it out in feet and inches per the following example:

```
Enter your height in inches: 69
Your height is: 5.0 feet, 9.0 inches
```

Exercise

Write a function called **`mileage`** that takes an integer, which represents the number of trips a driver has taken in his/her car during the previous week. In a loop, get the mileage for each trip from the user (use the `input` function), keeping track of the total. At the end, print out the total distance the user drove in the past week.

Exercise

Write a function called **`partial_sum`** that takes an integer `n` and returns the following partial sum:

$$\frac{1}{2^0} + \frac{1}{2^1} + \frac{1}{2^2} + \cdots + \frac{1}{2^n}$$

Exercise

Write a function called **`random_color_string`** that takes no arguments and returns a randomly generated color string. A color string consists of the pound sign (#) followed by six hexadecimal digits (`0-9`, `A-F`). Recall that `random.randrange(n)` returns an integer between `0` and `n - 1`, inclusive. Make a string with the hex digits in it and then use string indexing on the digit string in a loop to build the color string. You will be accumulating values into a variable (building a string), so you will need to initialize the new string before the loop starts. I suggest starting with the string `'#'`.